Blockchain Technology (BCT)

Practicals 2, 3 and 4

Study and Execution Guide

Theory, line-by-line code explanation and step-by-step execution on a brand-new machine

Practical	Topic	File used
BCT2	Create your own wallet using MetaMask for crypto transactions	No code, MetaMask + Sepolia testnet
BCT3	Bank account smart contract: deposit, withdraw, show balance	3_Bank_account.sol
BCT4	Student data contract: structures, arrays, fallback; observe fee and gas	4_Student.sol

How to use this guide. Read Part 0 once for the Solidity and Ethereum basics. Then each practical has its own theory, a line-by-line walkthrough of the code, and a numbered execution guide that starts from a machine with nothing installed. The viva questions at the end of each part are the ones examiners usually ask.

> **NOTE:** Written for a developer who knows Java and Spring but is new to Solidity. Gas figures in the reference tables were measured on a local simulator running the exact contracts. Screens in MetaMask, Remix and faucets change over time, so wording of buttons may differ slightly. Prepared November 2026.

Contents

Part 0: Read This First - Blockchain and Solidity for a Developer 3

0.1 The big picture . . . 3
0.2 Life cycle of a .sol file . . . 3
0.3 Concept map for a Java / Spring developer . . . 3
0.4 Units, gas and the fee formula . . . 4
0.5 Account types . . . 4
0.6 Solidity syntax cheat-sheet . . . 5

Part 1: BCT2 - Create Your Own Wallet Using MetaMask 6

1.1 Theory . . . 6
1.2 What you need on a brand-new machine . . . 6
1.3 Step-by-step execution . . . 7
1.4 Evidence checklist for your journal . . . 8
1.5 Troubleshooting . . . 8
1.6 Viva questions . . . 9

Part 2: BCT3 - Bank Account Smart Contract (Deposit, Withdraw, Show Balance) 10

2.1 Theory and design . . . 10
2.2 Line-by-line explanation of 3_Bank_account.sol . . . 10
2.3 Execution on a brand-new machine . . . 12
2.4 Reference gas figures . . . 14
2.5 Troubleshooting . . . 14
2.6 Viva questions . . . 15

Part 3: BCT4 - Student Data Contract (Structures, Arrays, Fallback) and Gas Observation 16

3.1 Theory . . . 16
3.2 Line-by-line explanation of 4_Student.sol . . . 17
3.3 Execution on a brand-new machine . . . 20
3.4 Reference gas figures for checking your results . . . 22
3.5 Troubleshooting . . . 23
3.6 Viva questions . . . 23

Appendix: Quick Reference 25

A. Exam-day checklist . . . 25
B. Unit converter . . . 25
C. Remix button colours . . . 25
D. Common revert messages in these contracts . . . 25
E. Useful links (type them yourself, do not follow ads) . . . 25

Part 0: Read This First - Blockchain and Solidity for a Developer

You already know classes, HashMaps, transactions and exceptions from Java and Spring. This part maps those ideas onto Ethereum and Solidity so the three practicals make sense before you touch any tool.

0.1 The big picture

A **blockchain** is a database that is copied on thousands of computers (called nodes) and is owned by nobody. New data is added in blocks. Every block holds a list of transactions and the hash of the previous block, so changing old data breaks the chain and is detected. **Ethereum** is a blockchain that can also run programs. Those programs are called **smart contracts**.

Everything you do on Ethereum is a **transaction** signed with your private key. A transaction can send ETH, create (deploy) a contract, or call a contract function. The network charges a fee for each transaction, called **gas**, paid in ETH. On a **test network** (testnet) the ETH is free and has no value, so you can practise safely.

Smart contracts are written in **Solidity**, in files that end with `.sol`. You compile the file into bytecode, send the bytecode to the network in a deployment transaction, and the network gives the contract a permanent address. After deployment the code cannot be changed. Anyone can call its public functions.

0.2 Life cycle of a .sol file

Stage	What happens	Tool in these practicals
1. Write	You write the contract in a .sol file.	Remix editor (browser)
2. Compile	The Solidity compiler (solc) converts the source to EVM bytecode and an ABI (JSON interface description).	Remix, Solidity Compiler tab
3. Deploy	A special transaction carries the bytecode. The network creates the contract and gives it an address. The constructor runs once.	Remix Deploy tab + MetaMask confirmation
4. Interact	You send **transactions** (change state, cost gas, need signature) or make **calls** (read-only, free, no signature).	Remix buttons under Deployed Contracts
5. Verify	You inspect the transaction, gas used, fee and event logs.	Remix terminal and Sepolia Etherscan

0.3 Concept map for a Java / Spring developer

Solidity / Ethereum	Closest thing you know	Important difference
contract	A singleton class instance	State lives on the chain forever. Code is immutable after deployment. It has its own address and its own ETH balance.
state variable	A field persisted to a database	Every write costs gas. Reading from outside is free.
mapping(K => V)	HashMap	No iteration, no size(). Every key 'exists' and returns a default (0, false, empty) if never written.
struct	POJO / record with only fields	No methods. Just a group of fields.
array T[]	ArrayList	push and pop exist. Index out of range reverts. Storage arrays are expensive.
address	User id / principal	20-byte account id (0x + 40 hex characters), derived from a public key.
msg.sender	Authenticated user in SecurityContext	Proved by a cryptographic signature. It cannot be faked.
msg.value	Money attached to the request	Amount of ETH sent with the call, in wei.

Solidity / Ethereum	Closest thing you know	Important difference
require(cond, 'msg')	if (!cond) throw new IllegalStateException(msg)	The revert rolls back every state change of the whole transaction, like a @Transactional rollback.
event / emit	Logger or message published to a topic	Stored in the transaction receipt. Cheap. Readable by off-chain apps, not by contracts.
gas	Metered billing per CPU and storage operation	The caller pays. If gas runs out the transaction fails and the fee is still charged.
view function	A read-only GET endpoint	Called from outside it needs no transaction and costs no fee.
payable	An endpoint that accepts money	Without payable, sending ETH to that function reverts.
ABI	OpenAPI spec / interface	JSON that tells tools how to encode a call and decode the result.
constructor	Constructor	Runs exactly once, at deployment. Cannot be called again.
(no null, no float)	-	Only integers. Money is stored in the smallest unit (wei). Every variable has a default zero value.

0.4 Units, gas and the fee formula

Unit	Value in wei	Use
wei	1	Smallest unit. Contracts work in wei.
gwei	10^9 wei	Gas prices are quoted in gwei.
ether (ETH)	10^{18} wei	What you see in MetaMask.

Transaction fee = gas used x effective gas price. Since the London upgrade the effective gas price is the network base fee plus a small priority tip. Example with an assumed price of 2 gwei: a plain ETH transfer uses 21,000 gas, so the fee is 21,000 x 2 gwei = 42,000 gwei = 0.000042 ETH.

Gas limit is the maximum you allow. **Gas used** is what the transaction really consumed. Unused gas is refunded. Rough cost guide (these figures are standard EVM rules):

Operation	Approximate gas
Base cost of any transaction	21,000
Extra for creating a contract	32,000
Calldata / deployment bytes	16 per non-zero byte, 4 per zero byte
Contract code stored on chain	200 per byte of runtime code
Writing a storage slot from zero to non-zero	about 22,100 (20,000 + 2,100 cold access)
Overwriting a storage slot that is already non-zero	about 5,000 (2,900 + 2,100 cold access)
Emitting an event	about 375 + 375 per topic + 8 per data byte

> **TIP:** This is why deployment is the most expensive step, why the first add of a record costs more than later ones, and why reading data from outside is free: only writes to the chain cost gas.

0.5 Account types

EOA (externally owned account) is controlled by a private key. Your MetaMask accounts are EOAs. Only EOAs can start a transaction. **Contract account** is controlled by its code. It has an address, storage and an ETH balance, but it only reacts when an EOA (or another contract) calls it.

0.6 Solidity syntax cheat-sheet

Construct	Example	Meaning
Comment	`// text`	Same as Java.
Licence tag	`// SPDX-License-Identifier: MIT`	Declares the licence. The compiler warns if it is missing.
Version pragma	`pragma solidity ^0.8.19;`	Allowed compiler versions: 0.8.19 up to (not including) 0.9.0.
Contract	`contract Name { ... }`	Like a class. One instance is created at deployment.
Integer types	`uint256, uint8, int`	Unsigned integers of 256 and 8 bits. uint8 holds 0 to 255. From 0.8 overflow reverts.
Other types	`bool, address, string, bytes`	address is a 20-byte account id. string is dynamic text. bytes is raw data.
Visibility	`public external internal private`	public: anyone and internal calls. external: only from outside. internal: this contract and children. private: this contract only.
State mutability	`view, pure, payable`	view: reads state, no writes. pure: touches no state. payable: can receive ETH.
Data location	`storage memory calldata`	storage: permanent chain storage (expensive). memory: temporary during the call. calldata: read-only call arguments (cheapest).
Globals	`msg.sender msg.value msg.data`	Caller address, wei sent with the call, raw call data.
Guard	`require(cond, 'msg')`	Reverts with a message when cond is false.
Event	`event X(...); emit X(...);`	Declares and writes a log entry.
Special functions	`constructor receive fallback`	Run at deployment, on plain ETH transfer, and on unknown calls.

Part 1: BCT2 - Create Your Own Wallet Using MetaMask

NOTE: Aim: Create your own wallet using MetaMask for crypto transactions.

WARNING: MetaMask, Remix and the faucets update their screens often. Button names in this guide are accurate as of November 2026 but may differ slightly on your day. The order of steps stays the same.

1.1 Theory

What is a crypto wallet?

A wallet does **not** store coins. Your balance is a record on the blockchain. The wallet stores your **keys**, shows the balance, and signs transactions. **MetaMask** is a free, non-custodial wallet that runs as a browser extension. Non-custodial means only you hold the keys, no company can reset or recover them.

Keys, address and recovery phrase

Item	What it is	Share it?
Private key	A secret 256-bit number. It signs transactions and proves ownership.	**Never**
Public key	Derived from the private key using elliptic-curve maths (secp256k1). One-way: you cannot go back.	Not needed
Address	The last 20 bytes of the Keccak-256 hash of the public key, written as 0x plus 40 hex characters.	Yes, like an account number
Secret Recovery Phrase	12 words (BIP-39 standard) that generate all your private keys. Anyone with it controls every account.	**Never**
Password	Only unlocks MetaMask on this one browser. Does not recreate the wallet elsewhere.	Never

Networks

The same address works on every Ethereum-compatible network, but balances are separate per network. **Ethereum Mainnet** uses real ETH (chain ID 1). **Sepolia** is a public **test network** (chain ID 11155111) with free test ETH from **faucets**. All practicals use Sepolia. If your lab sheet names another testnet, use that one.

Life of a transaction

Stage	What happens
1. Create	You choose recipient, amount, and gas settings. MetaMask adds a **nonce** (per-account counter that fixes the order and stops replays).
2. Sign	MetaMask signs the data with your private key when you press Confirm.
3. Broadcast	The signed transaction is sent to network nodes and waits in the mempool.
4. Include	A validator puts it in a block. MetaMask shows Confirmed.
5. Final	A few minutes later the block is finalised. One confirmation is enough for the lab.

1.2 What you need on a brand-new machine

- A Windows, macOS or Linux computer with internet.
- A browser that supports MetaMask: Chrome, Brave, Edge or Firefox. These steps use Chrome.
- Pen and paper for the 12-word recovery phrase.

- A Google or GitHub account may be needed by some faucets (see Step 10).

1.3 Step-by-step execution

Step 1: Install a browser (skip if you already have one)

- Open any browser, search for 'Download Chrome', open the official google.com/chrome page, install and open it.
- Edge, Brave and Firefox also work.

Step 2: Open the official MetaMask site

- In the address bar type **metamask.io** and press Enter. Do not use sponsored search results: fake MetaMask sites exist.
- Click **Download**. It opens the browser extension store page for MetaMask.
- Click **Add to Chrome**, then **Add extension**.

Step 3: Pin the extension

- Click the puzzle-piece icon in the toolbar, then the pin icon next to MetaMask. The fox icon now stays visible.

Step 4: Start wallet creation

- MetaMask opens a welcome tab. Accept the terms, click **Create a new wallet**.
- If asked to share usage data, choose **No thanks**.

Step 5: Create a password

- Choose a password of at least 8 characters. Tick the 'I understand' box. Click **Create new wallet**.
- This password only protects MetaMask on this browser.

Step 6: Secure the Secret Recovery Phrase

- Click **Secure my wallet** and reveal the 12 words.
- Write the 12 words **in order** on paper. Do not take a screenshot, do not save in a cloud note, do not share.
- Confirm the phrase by choosing the missing words, then click Done.

Step 7: Look at your first account

- MetaMask shows **Account 1**. Click the address (starts with 0x) to copy it.
- Screenshot 1: Account 1 with its address.

Step 8: Switch to the Sepolia test network

- Click the network selector at the top (it says Ethereum Mainnet or similar).
- If Sepolia is missing, turn on **Show test networks** in that list (or Settings, Advanced, Show test networks).
- Select **Sepolia**. The balance shows 0 SepoliaETH.

Step 9: Get free test ETH from a faucet

- A faucet is a website that sends free test ETH to an address. Open one in a new tab, paste your address, claim.
- Options (requirements change, if one refuses try another): Google Cloud Web3 faucet (Google login), Alchemy Sepolia faucet at alchemy.com/faucets/ethereum-sepolia (free Alchemy account, may require a small mainnet balance), Chainlink faucet, QuickNode faucet, and the pk910 Sepolia PoW faucet (no account, you mine in the browser for a few minutes and keep the tab open).
- Wait 1 to 5 minutes. About 0.05 SepoliaETH is enough for all three practicals.
- Screenshot 2: balance after the faucet.

Step 10: Create a second account

- Click the account name at the top, choose **Add account or hardware wallet**, then **Add a new Ethereum account**, name it Account 2.
- Copy the new address. Screenshot 3.

Step 11: Send a transaction

- Switch to Account 1. Click **Send**. Choose SepoliaETH. Paste the address of Account 2. Enter 0.001.
- Click Continue and read the **network fee**. Screenshot 4. Click **Confirm**.

Step 12: Watch it confirm

- Open the **Activity** tab. The status goes Pending then Confirmed within seconds to a minute. Screenshot 5.

Step 13: Open it on the block explorer

- Click the transaction, then **View on block explorer**. It opens Sepolia Etherscan (sepolia.etherscan.io).
- Write down: Transaction Hash, Status (Success), Block, From, To, Value, Transaction Fee, Gas Price, Gas Used. Screenshot 6.

Step 14: Verify the balances

- Account 2 gained 0.001 SepoliaETH. Account 1 lost 0.001 plus the fee.

Step 15: Lock the wallet when you leave

- Open MetaMask, click the menu, choose **Lock**. Never leave an unlocked wallet on a shared lab PC.

TIP: Using the same wallet on another machine. Faucets limit how often you can claim. On a new machine install MetaMask (Steps 1 to 3), choose **Import an existing wallet**, enter your 12 words, set a new password and select Sepolia. Your accounts and test ETH reappear because the balance lives on the blockchain, not in the browser. Use a recovery phrase made only for practice. Never type a phrase that protects real money into a shared lab PC.

1.4 Evidence checklist for your journal

#	Screenshot	Shows
1	MetaMask home, Account 1	Wallet created, address visible
2	Balance after faucet	Sepolia network selected, balance above 0
3	Account 2 created	Second address
4	Send confirmation screen	Recipient, amount, network fee
5	Activity tab	Status Confirmed
6	Etherscan transaction page	Hash, status, from, to, value, fee

WARNING: Never include the Secret Recovery Phrase in any screenshot or in your journal.

1.5 Troubleshooting

Problem	Likely cause	Fix
Sepolia not in the network list	Test networks are hidden	Enable Show test networks, or Settings, Advanced.
Balance still 0 after faucet	Wrong network selected, or faucet still processing	Check Sepolia is selected, wait, look up your address on sepolia.etherscan.io, or use another faucet.
Faucet refuses the request	Anti-spam rules (login or mainnet balance needed)	Try another faucet or the PoW faucet.
Transaction pending for long	Fee too low	Use Speed up in Activity, or wait.
Cannot see the extension	Not pinned	Puzzle icon, then pin MetaMask.
Forgot password	-	Use Forgot password and your 12 words. Without the phrase the wallet cannot be recovered.
Cannot install extensions on lab PC	Browser policy	Ask the lab admin or use your own laptop.

1.6 Viva questions

Question	Answer
What is MetaMask?	A non-custodial browser wallet. It stores keys, signs transactions and connects to Ethereum apps.
Does a wallet store coins?	No. It stores keys. Balances are on the blockchain.
Difference between private key and address?	The private key is secret and signs. The address is derived from the public key and is safe to share.
What is the recovery phrase?	12 words that generate all private keys. Lose it and the funds are gone, leak it and anyone can steal them.
What is a testnet?	A practice network with valueless ETH that works like the real chain.
What is a faucet?	A website that gives free test ETH.
What is gas?	The fee for computation and storage on the network, paid in ETH.
What is a nonce?	A per-account counter that orders transactions and prevents replay.
Custodial versus non-custodial?	Custodial: a company holds your keys (an exchange). Non-custodial: you do (MetaMask).

Part 2: BCT3 - Bank Account Smart Contract (Deposit, Withdraw, Show Balance)

> **NOTE: Aim:** Write a smart contract on a test network for the bank account of a customer with these operations: deposit money, withdraw money, show balance.

Source file: `3_Bank_account.sol` (contract `BankAccount`). It compiles without errors. The compiler gives one harmless warning about the deprecated `transfer`.

2.1 Theory and design

The contract works like a shared vault. Customers send test ETH into the contract. Because Ethereum contracts cannot store a table of customers the way a database does, the contract keeps a **ledger** in a mapping (address to balance). The real ETH sits in the contract's own balance. The two must always agree: the contract's ETH balance equals the sum of all ledger balances.

Requirement	Function	ETH moves	State change	Reverts when
Open account	`createAccount()`	Customer to contract (optional amount)	isUser = true, balance = value	Account already exists
Deposit money	`deposit()`	Customer to contract	balance increases	No account, or value is 0
Withdraw money	`withdraw(amount)`	Contract to customer	balance decreases	No account, or balance too low
Show balance	`showBalance()`	None (read only)	None	No account

Design points to mention in the viva

- **mapping** gives O(1) lookup by address, an array would need a loop (expensive).
- **msg.sender** identifies the customer. Nobody can touch another person's balance because the signature proves who is calling.
- **Checks-effects-interactions:** check conditions, update the ledger, then send ETH last.
- **Events** record every action cheaply for auditing and for front-ends.
- There is no receive or fallback function, so ETH sent straight to the contract address (without calling deposit) reverts. This prevents money getting stuck without a ledger entry.
- All chain data is public. 'private' only hides a variable from other contracts' code.

2.2 Line-by-line explanation of 3_Bank_account.sol

#	Code	What it does
1	`// SPDX-License-Identifier: UNLICENSED`	A comment (*//* as in Java). The SPDX tag states the licence of the code. The compiler prints a warning if it is missing. UNLICENSED means no licence is granted. Not executed.
2	`pragma solidity ^0.8.19;`	Compiler version rule: any 0.8.x version from 0.8.19 upwards, below 0.9.0. Version 0.8 or later matters because arithmetic overflow and underflow now **revert** automatically (older versions wrapped silently).
4	`contract BankAccount {`	Declares the contract. Think of a class, but exactly one instance is created at deployment. It lives forever at its own address with its own storage and its own ETH balance.

#	Code	What it does
5	`  mapping(address => uint256) private balances;`	**State variable** (stored permanently on the chain). A mapping is a hash map from account address to an unsigned 256-bit integer. It holds each customer's balance in **wei**. A key that was never written returns 0. A mapping cannot be looped over and has no size. **private** blocks other contracts from reading it by code, but anyone can still read raw chain data, so private is not secret.
6	`  mapping(address => bool) private isUser;`	Second map: has this address opened an account? It is needed because a balance of 0 cannot tell 'no account' apart from 'account with zero balance'. Default is false.
8	`  event AccountCreated(address user, uint256 amount);`	Declares an **event** type: a log entry written into the transaction receipt. It is not storage, is cheap, and is what Etherscan, Remix and front-end apps read. Fired when an account is opened.
9	`  event Deposit(address user, uint256 amount);`	Event for every deposit: who deposited and how much (wei).
10	`  event Withdraw(address user, uint256 amount);`	Event for every withdrawal: who withdrew and how much (wei).
12	`  function createAccount() public payable {`	Opens an account for the caller. **public**: callable from outside and from inside. **payable**: the call may carry ETH (**msg.value**). Without payable, any call with ETH attached would revert. This is a state-changing function, so it needs a transaction and gas.
13	`    require(!isUser[msg.sender], "Account exists");`	**msg.sender** is the address that signed the transaction, like the authenticated user in Spring. **require(condition, message)** reverts the whole transaction if the condition is false: all state changes are undone and the message is returned. Here it blocks opening a second account for the same address.
14	`    isUser[msg.sender] = true;`	Marks the caller as a customer. This is a storage write. A slot going from zero to non-zero costs about 22,100 gas.
15	`    balances[msg.sender] = msg.value;`	**msg.value** is the amount of wei sent with this call. The EVM has already added it to the contract's own ETH balance before this line runs. Here the contract records it as the customer's balance (it can be 0).
16	`    emit AccountCreated(msg.sender, msg.value);`	Writes the AccountCreated event into the log.
17	`  }`	Closes createAccount.
19	`  function deposit() public payable {`	Adds money to an existing account. **payable** because ETH arrives with the call.
20	`    require(isUser[msg.sender], "No account");`	Guard: the caller must already have an account, otherwise revert with 'No account'.
21	`    require(msg.value > 0, "Amount > 0");`	Guard: reject zero-value deposits (they would waste gas and do nothing).
22	`    balances[msg.sender] += msg.value;`	Increases the caller's balance. In 0.8 the addition is overflow-checked automatically.
23	`    emit Deposit(msg.sender, msg.value);`	Writes the Deposit event.
24	`  }`	Closes deposit.
26	`  function withdraw(uint256 amount) public {`	Sends money back to the caller. Not payable: no ETH comes in. The argument **amount** is in **wei** (1 ETH = 10^18 wei), so 0.005 ETH is entered as 5000000000000000.
27	`    require(isUser[msg.sender], "No account");`	Guard: only account holders can withdraw.
28	`    require(balances[msg.sender] >= amount, "Low balance");`	Guard: the caller cannot withdraw more than the recorded balance.

#	Code	What it does
29	`balances[msg.sender] -= amount;`	**Effect first:** the ledger is reduced *before* any ETH leaves the contract. This is the **checks-effects-interactions** pattern. It stops a re-entrancy attack, where a malicious contract calls withdraw again from inside the transfer and drains the bank.
30	`payable(msg.sender).transfer(amount);`	**Interaction last.** msg.sender is a plain address. **payable(...)** converts it into an address that can receive ETH. **.transfer(amount)** sends *amount* wei from the contract's balance to the caller. It forwards only 2300 gas and reverts if sending fails. The compiler shows a warning because transfer is deprecated. Modern code uses `call{value: amount}("")` and checks the result. For MetaMask wallets this line works correctly.
31	`emit Withdraw(msg.sender, amount);`	Writes the Withdraw event.
32	`}`	Closes withdraw.
34	`function showBalance() public view returns (uint256) {`	**view** means it only reads state. When called from outside (Remix blue button, or an eth_call) it runs on a node, needs **no transaction, no gas fee and no MetaMask popup**. **returns (uint256)** declares the return type. Because it uses msg.sender, each caller only sees their own balance (Remix passes the selected account as the sender).
35	`require(isUser[msg.sender], "No account");`	Guard: no account means there is nothing to show.
36	`return balances[msg.sender];`	Returns the caller's balance in wei.
37	`}`	Closes showBalance.
38	`}`	Closes the contract.

2.3 Execution on a brand-new machine

Phase A: Fresh machine setup (do this first, every time)

A 1: Open a supported browser

- Chrome, Brave, Edge or Firefox. Install Chrome from google.com/chrome if the machine has none.

A 2: Install MetaMask

- Go to **metamask.io** by typing it yourself, click Download, then Add to Chrome, Add extension. Pin it with the puzzle icon.

A 3: Create or restore the wallet

- **New wallet:** Create a new wallet, set a password, write the 12-word recovery phrase on paper, confirm it.
- **Existing wallet:** choose Import an existing wallet, enter your 12 words, set a new password. Your Sepolia balance returns automatically.

A 4: Select the Sepolia test network

- Network selector at the top. Enable Show test networks if Sepolia is hidden. Choose Sepolia.

A 5: Get test ETH if the balance is 0

- Copy the address (click it). Claim from a faucet such as Google Cloud Web3 faucet, Alchemy Sepolia faucet or the pk910 PoW faucet. Wait until MetaMask shows the balance. Keep at least 0.05 SepoliaETH.

A 6: Check before moving on

- MetaMask is unlocked, network says Sepolia, balance is above 0. If not, fix this first (see Part 1, Troubleshooting).

Phase B: Compile and deploy in Remix

B 1: Open Remix

- In the browser go to **remix.ethereum.org**. Accept or dismiss the first-run banner. Remix needs no installation and no account.
- The left side has an icon bar: File Explorer, Search, Solidity Compiler, Deploy and Run. The bottom panel is the Terminal.

B 2: Add the file

- Click the File Explorer icon. In the default workspace, right-click the **contracts** folder and choose **New File**. Name it `BankAccount.sol`.
- Open the file and paste the full contents of 3_Bank_account.sol. (Alternative: use the Upload files icon in File Explorer and pick the .sol file.)

B 3: Compile

- Click the **Solidity Compiler** icon.
- Set **Compiler** to 0.8.19 or any newer 0.8.x version.
- Click **Compile BankAccount.sol**. A green tick appears on the icon. One yellow warning about 'transfer' is normal. Red errors mean the file was pasted incorrectly.

B 4: Connect Remix to MetaMask

- Click the **Deploy and Run Transactions** icon.
- Set **Environment** to **Injected Provider - MetaMask**. A MetaMask popup asks to connect. Click Next, then Connect.
- Remix now shows the network (Sepolia, chain id 11155111) and your account address with its balance. If the network is wrong, change it in MetaMask and refresh Remix.

B 5: Deploy

- In the **Contract** drop-down choose **BankAccount**. Keep **Value** at 0. Click the orange **Deploy** button.
- A MetaMask popup shows the network fee. Screenshot it, then click **Confirm**.
- Wait 10 to 30 seconds. The Terminal shows the creation transaction succeeded. The contract appears under **Deployed Contracts** with an address. Copy that address.

B 6: Open the contract

- Click the arrow next to the deployed contract to expand it. You see four buttons: createAccount, deposit, withdraw, showBalance. Red = payable function (accepts ETH). Orange = normal state-changing function. Blue = read-only (view).

Phase C: Test the three required operations

> **WARNING:** Remix keeps the **Value** field (and its unit) for every later transaction. Set it before each payable call and **set it back to 0** afterwards. If Value stays above 0 when you press withdraw, the call fails because withdraw is not payable.

#	Action in Remix	Expected result
1	Value = 0.01, unit **Ether**. Click **createAccount**. Confirm in MetaMask. Set Value back to 0.	Transaction succeeds. In the Terminal, logs show AccountCreated.
2	Click blue **showBalance**.	Returns **10000000000000000** (0.01 ETH in wei). No MetaMask popup, no fee.
3	Value = 0.005 Ether. Click **deposit**. Confirm. Set Value to 0.	Transaction succeeds. Deposit event logged.
4	Click **showBalance**.	Returns **15000000000000000**.
5	In the withdraw box type **5000000000000000** (0.005 ETH). Click **withdraw**. Confirm.	Succeeds. MetaMask balance rises by 0.005 ETH minus the fee. Withdraw event logged.
6	Click **showBalance**.	Returns **10000000000000000**.

Optional negative tests (good for the viva). MetaMask may warn that the transaction is expected to fail. That is the point of the test.

Action	Expected revert message
Click createAccount again	Account exists
withdraw with 99999999999999999999	Low balance
deposit with Value 0	Amount > 0
Call deposit from a new, second MetaMask account	No account

Phase D: Collect evidence and observe the fee

D 1: Remix Terminal

- Click the small arrow on a transaction line to expand it. Note status, hash, gas, transaction cost and execution cost, decoded input and logs.

D 2: Etherscan

- In MetaMask Activity open a transaction, click View on block explorer. Note Transaction Fee, Gas Price and Gas Used.

D 3: Contract page

- Paste the contract address into sepolia.etherscan.io to show the deployed contract and its transactions.

D 4: Screenshots

- Compile success, deployment popup, each function result, showBalance results, Etherscan page of the deployment transaction.

2.4 Reference gas figures

Measured by running this exact compiled contract on a local Ethereum test simulator. Your numbers on Sepolia will be close but not identical. Fee example uses an **assumed** price of 2 gwei, read the real price from your own transaction.

Operation	Gas used	Fee at 2 gwei (ETH)
Deploy BankAccount	570,790	0.00114158
createAccount (first time)	67,491	0.000134982
deposit	30,375	0.00006075
withdraw	37,886	0.000075772
showBalance (view call)	0, free	0

Compare: a plain ETH transfer costs 21,000 gas. Deployment is the most expensive because it pays for storing about 2.4 KB of contract code at 200 gas per byte.

2.5 Troubleshooting

Problem	Cause	Fix
Remix shows red compile errors	Partial paste, or compiler below 0.8.19	Paste the entire file, select compiler 0.8.19 or newer.
Environment list has no MetaMask option	MetaMask not installed, locked or not detected	Unlock MetaMask, refresh Remix, check the extension is enabled for this site.
Remix shows the wrong network	MetaMask is on another network	Select Sepolia in MetaMask, refresh Remix.
Deploy popup says insufficient funds	No test ETH	Use a faucet (Part 1, Step 10).

Problem	Cause	Fix
Transaction fails: No account	createAccount not called with this account	Call createAccount first from the same account.
withdraw fails although balance is enough	Value field still above 0	Set Value to 0.
Entered withdraw amount 0.005 and it failed	Input is in wei, not ETH	Type 5000000000000000.
showBalance reverts	Selected account has no bank account	Use the account that called createAccount.
Transaction stuck pending	Low fee or slow network	Wait, or Speed up in MetaMask Activity.

2.6 Viva questions

Question	Answer
What is a smart contract?	A program stored on the blockchain that runs exactly as written when called. Its code cannot be changed after deployment.
Why is createAccount and deposit payable?	Because ETH arrives with the call. Non-payable functions revert if ETH is attached.
Difference between view and a normal function?	view only reads state, so it is free to call from outside and needs no transaction. A normal function changes state and needs a transaction and gas.
Why does showBalance cost nothing?	It is a read executed locally by a node, nothing is written to the chain.
What is msg.sender and msg.value?	The caller's address and the wei sent with the call.
What happens when require fails?	The transaction reverts, all state changes are undone, the message is returned. Gas already used is still charged, unused gas is refunded.
Why reduce the balance before sending ETH?	To prevent a re-entrancy attack.
What is wei?	The smallest ETH unit, 1 ETH = 10^18 wei.
Where is the real money kept?	In the contract's own ETH balance. The mapping is only a ledger of who owns how much.
What if someone sends ETH directly to the contract?	It reverts because there is no receive or fallback function.
transfer vs send vs call?	transfer reverts on failure and forwards 2300 gas. send returns false on failure. call forwards all gas, returns a success flag, and is the recommended modern way.
Why deploy on a testnet?	Free test ETH, same behaviour as mainnet, no financial risk.

Part 3: BCT4 - Student Data Contract (Structures, Arrays, Fallback) and Gas Observation

> **NOTE: Aim:** Write a Solidity program to create student data using structures, arrays and a fallback function. Deploy it as a smart contract on Ethereum and observe the transaction fee and gas values.

Source file: `4_Student.sol` (contract `StudentManager`). It compiles with no errors and no warnings.

Required construct	Where it is in the code
Structure	`struct Student { id, name, age, course }`
Array	`Student[] private students;` with push, pop, length and index
Fallback	`fallback() external payable` (and `receive()` for plain ETH)
Gas and fee observation	Deployment transaction and every function call, read from Remix, MetaMask and Etherscan

3.1 Theory

Structures and arrays

A **struct** groups related fields into one type, so a student becomes one value instead of four unrelated variables. A **dynamic array** (`Student[]`) is a growable list of such structs stored on the chain. Arrays give order and easy listing, but searching by id would need a loop, which costs gas. The contract therefore keeps a mapping `idx` from id to position, so lookups are one step.

Storage, memory and calldata

Location	Lifetime	Cost	Used in this file for
storage	Permanent, on the chain	Highest	students, idx, counters
memory	Only during the call	Low	Return values such as Student memory, lastStudent copy
calldata	Read-only call arguments	Lowest	string parameters of addStudent and updateStudent

The +1 index trick and swap-and-pop deletion

The mapping stores **position + 1** so that 0 means 'not present'. To delete an element from the middle of an array cheaply, the last element is copied into the hole and the last slot is popped. Example, removing id 1:

	Array contents (index: student)	idx mapping (id : stored value)
Before	0: id 1 Vedant, 1: id 2 Aman, 2: id 3 Riya	1:1, 2:2, 3:3
Steps	Copy id 3 into index 0, set idx[3] = 1, pop the last slot, set idx[1] = 0	
After	0: id 3 Riya, 1: id 2 Aman	1:0 (removed), 2:2, 3:1

receive() versus fallback()

When a call reaches the contract, Solidity decides which code runs:

Call data	ETH attached	What runs
Matches a function selector, e.g. addStudent(...)	Any (if function is payable)	That function
Empty	Yes	receive() if it exists, else payable fallback()
Empty	No	receive() if it exists, else fallback()

Call data	ETH attached	What runs
Not empty, matches nothing (e.g. 0x1234)	Any	fallback() if it exists, otherwise the call reverts

Real uses of fallback: catching mistaken calls, proxy contracts that forward calls, and logging unexpected interactions.

Events and indexed parameters

Events are cheap logs. **indexed** parameters become topics that can be searched. Public state variables get automatic getter functions.

Gas theory for the observation task

- **Fee = gas used x effective gas price.** Gas used depends on what the code does. Gas price depends on network demand.
- **Deployment** pays 21,000 base + 32,000 for creation + 16 gas per non-zero byte of the bytecode + 200 gas per byte of runtime code stored. Larger contract means higher cost.
- **First write to an empty storage slot** costs about 22,100 gas. **Overwriting a non-zero slot** costs about 5,000. So the first addStudent costs more than later ones, and the first deposit costs more than later ones.
- **Read functions** (getStudent, getAllStudents, studentCount) called from outside cost no gas and need no signature.
- A failed transaction still pays for the gas it used before reverting.

3.2 Line-by-line explanation of 4_Student.sol

#	Code	What it does
1	`// SPDX-License-Identifier: MIT`	Comment with the licence tag. MIT is an open-source licence. Not executed.
2	`pragma solidity ^0.8.19;`	Compiler version rule: 0.8.19 or newer inside 0.8.x. Overflow checks are built in from 0.8.
4	`contract StudentManager {`	Declares the contract. One instance is created at deployment and lives at its own address.
5	`  struct Student {`	**Structure (requirement 1).** A **struct** is a custom record type, like a Java class that has only fields. Declaring a struct uses no storage by itself.
6	`    uint256 id;`	Unsigned 256-bit integer: the student id.
7	`    string name;`	Dynamic UTF-8 text. Strings are costly to store. A string of 31 bytes or less fits in one storage slot.
8	`    uint8 age;`	8-bit unsigned integer, range 0 to 255. A value outside the range is rejected when the call data is decoded.
9	`    string course;`	Course name, stored as text.
10	`  }`	Ends the struct.
12	`  Student[] private students;`	**Array (requirement 2).** A dynamic array of Student structs held in **storage**. It supports push, pop, length and indexing. An index out of range reverts. **private** means other contracts cannot read it directly, the contract exposes getter functions instead.
13	`  mapping(uint256 => uint256) private idx;`	Lookup table: student id to (array position + 1). It avoids looping over the array to find a student, which would cost gas. The +1 trick exists because a mapping returns 0 for unknown keys, so 0 can mean 'not found'.
14	`  uint256 public studentCount;`	Counter used to create new ids. **public** makes the compiler generate a free getter function `studentCount()`. The counter only goes up, so ids are never reused, even after a delete.

#	Code	What it does
16	`  uint256 public depositsCount;`	How many times ETH has been received through deposit(), receive() or fallback(). Public getter generated.
17	`  uint256 public lastDepositAmount;`	Amount in wei of the most recent ETH received.
18	`  address public lastSender;`	**address** is the 20-byte account id. Stores who sent ETH most recently.
20	`  event StudentAdded(uint256 indexed id, string name);`	Event declaration. **indexed** parameters are stored as searchable topics (maximum 3 per event). The non-indexed name goes into the log data.
21	`  event StudentUpdated(uint256 indexed id);`	Event for updates.
22	`  event StudentRemoved(uint256 indexed id);`	Event for removals.
23	`  event Received(address indexed from, uint256 amount);`	Event used by deposit() and receive(). You see it in the logs when ETH arrives with empty call data or through deposit().
24	`  event FallbackCalled(address indexed from, uint256 amount, bytes data);`	Event used only by fallback(). It also logs the raw call data (**bytes data**). In the demonstration you will see 0x1234 here, which proves the fallback ran.
26	`  constructor() {`	Runs **exactly once**, inside the deployment transaction. It can never be called again.
27	`    studentCount = 0;`	Sets the counter to 0. This is redundant because storage already defaults to 0, but it makes the intent clear.
28	`  }`	Ends the constructor.
30	`  function addStudent(string calldata _name, uint8 _age, string calldata _course) external {`	Creates a student. **external**: callable only from outside the contract (a transaction), slightly cheaper for large arguments. **calldata**: read-only area where the call arguments already live, the cheapest place for string parameters. The leading underscore in _name is a naming habit that avoids clashing with state variables. This writes to storage, so it needs a transaction and gas.
31	`    studentCount += 1;`	Increase the counter (storage write).
32	`    uint256 newId = studentCount;`	Local variable (not stored on the chain) that holds the new id.
33	`    students.push(Student({id: newId, name: _name, age: _age, course: _course}));`	Builds a Student struct using named fields and **appends it to the array**. This is the most expensive line: it writes several fresh storage slots (id, name, age, course and the array length).
34	`    idx[newId] = students.length;`	After the push, **length** equals the position of the new element counted from 1. Storing that gives the +1 convention used by the lookup.
35	`    emit StudentAdded(newId, _name);`	Logs the event.
36	`  }`	Ends addStudent.
38	`  function getStudent(uint256 _id) public view returns (Student memory) {`	Read function. **view**: no state change, free when called from outside. It returns a struct. **memory** means a temporary copy is returned to the caller. Remix displays it as a tuple.
39	`    uint256 i = idx[_id];`	Look up the 1-based position of this id (0 if missing).
40	`    require(i != 0, "Student not found");`	Guard: revert with a message if the id does not exist or was removed.
41	`    return students[i - 1];`	Convert the 1-based position back to a real array index (subtract 1) and return that struct.
42	`  }`	Ends getStudent.
44	`  function getAllStudents() external view returns (Student[] memory) {`	Returns the whole array. The array is copied from storage to memory. Fine for small data, because it is a free call from outside. For huge arrays this pattern is avoided.
45	`    return students;`	Returns every stored student.
46	`  }`	Ends getAllStudents.

#	Code	What it does
48	`  function updateStudent(uint256 _id, string calldata _name, uint8 _age, string calldata _course) external {`	Overwrites the data of an existing student. State-changing, so it needs a transaction.
49	`    uint256 i = idx[_id];`	Find the position.
50	`    require(i != 0, "Student not found");`	Guard against unknown ids.
51	`    Student storage s = students[i - 1];`	**storage** pointer: s is a *reference* to the array element that lives on the chain, not a copy. Changes made through s change the stored data. With **memory** here the edits would be lost because they would change only a temporary copy.
52	`    s.name = _name;`	Overwrite the name in storage.
53	`    s.age = _age;`	Overwrite the age.
54	`    s.course = _course;`	Overwrite the course. Overwriting existing non-zero slots is cheaper than the first write.
55	`    emit StudentUpdated(_id);`	Logs the event.
56	`  }`	Ends updateStudent.
58	`  function removeStudent(uint256 _id) external {`	Deletes a student using the **swap and pop** technique, which costs the same amount of gas no matter how large the array is. Order of the array is not preserved.
59	`    uint256 i = idx[_id];`	Find the 1-based position.
60	`    require(i != 0, "Student not found");`	Guard.
61	`    uint256 arrayIndex = i - 1;`	Real array index of the student to delete.
62	`    uint256 lastIndex = students.length - 1;`	Index of the last element. Safe here because the require above proves the array is not empty.
63	`    if (arrayIndex != lastIndex) {`	If the student to delete is already the last one, nothing needs to move. Otherwise the hole must be filled.
64	`      Student memory lastStudent = students[lastIndex];`	Copy the last element into memory.
65	`      students[arrayIndex] = lastStudent;`	Overwrite the deleted slot with that last element.
66	`      idx[lastStudent.id] = arrayIndex + 1;`	The moved student now has a new position, so update its lookup entry.
67	`    }`	Ends the if block.
68	`    students.pop();`	Removes the last array element and clears its storage (this gives a gas refund).
69	`    idx[_id] = 0;`	Marks the deleted id as 'not found'.
70	`    emit StudentRemoved(_id);`	Logs the event.
71	`  }`	Ends removeStudent.
73	`  function deposit() external payable {`	A normal named function that accepts ETH. **payable** allows the call to carry value. The caller must use the exact name deposit() and attach Value in Remix.
74	`    require(msg.value > 0, "Send ETH");`	Guard: refuse calls with no ETH.
75	`    depositsCount += 1;`	Count the deposit.
76	`    lastDepositAmount = msg.value;`	Remember the amount in wei.
77	`    lastSender = msg.sender;`	Remember who sent it.
78	`    emit Received(msg.sender, msg.value);`	Logs a Received event.

#	Code	What it does
79	`  }`	Ends deposit.
81	`  receive() external payable {`	**Special function.** It has no name, no 'function' keyword, no arguments and no return value. It runs when the contract is sent **plain ETH with empty call data** (for example a MetaMask Send to the contract address). A contract can have at most one receive function. It must be external and payable.
82	`    depositsCount += 1;`	Same bookkeeping as deposit().
83	`    lastDepositAmount = msg.value;`	Store the amount.
84	`    lastSender = msg.sender;`	Store the sender.
85	`    emit Received(msg.sender, msg.value);`	Logs Received. This is how you tell that receive() ran.
86	`  }`	Ends receive.
88	`  fallback() external payable {`	**Fallback function (requirement 3).** Runs when the call data matches **no function** of the contract. It also runs when ETH arrives with empty call data and there is no receive function. It is payable here, so it can accept ETH as well. Only one fallback is allowed. Without any fallback an unknown call reverts.
89	`    depositsCount += 1;`	Same bookkeeping.
90	`    lastDepositAmount = msg.value;`	Store the amount (0 if no ETH came with the call).
91	`    lastSender = msg.sender;`	Store the sender.
92	`    emit FallbackCalled(msg.sender, msg.value, msg.data);`	Logs FallbackCalled. **msg.data** is the complete raw call data of the call that did not match any function.
93	`  }`	Ends fallback.
95	`  function getStudentsLength() external view returns (uint256) {`	Free read function returning the number of stored students.
96	`    return students.length;`	Array length.
97	`  }`	Ends getStudentsLength.
98	`}`	Ends the contract.

3.3 Execution on a brand-new machine

Phase A: Fresh machine setup (do this first, every time)

A 1: Open a supported browser

- Chrome, Brave, Edge or Firefox. Install Chrome from google.com/chrome if the machine has none.

A 2: Install MetaMask

- Go to **metamask.io** by typing it yourself, click Download, then Add to Chrome, Add extension. Pin it with the puzzle icon.

A 3: Create or restore the wallet

- **New wallet:** Create a new wallet, set a password, write the 12-word recovery phrase on paper, confirm it.
- **Existing wallet:** choose Import an existing wallet, enter your 12 words, set a new password. Your Sepolia balance returns automatically.

A 4: Select the Sepolia test network

- Network selector at the top. Enable Show test networks if Sepolia is hidden. Choose Sepolia.

A 5: Get test ETH if the balance is 0

- Copy the address (click it). Claim from a faucet such as Google Cloud Web3 faucet, Alchemy Sepolia faucet or the pk910 PoW faucet. Wait until MetaMask shows the balance. Keep at least 0.05 SepoliaETH.

A 6: Check before moving on

- MetaMask is unlocked, network says Sepolia, balance is above 0. If not, fix this first (see Part 1, Troubleshooting).

Phase B: Compile and deploy in Remix

B 1: Open Remix

- Go to **remix.ethereum.org**. Dismiss the first-run banner.

B 2: Add the file

- File Explorer icon. Right-click the **contracts** folder, New File, name it `StudentManager.sol`.
- Paste the full contents of 4_Student.sol (or use the Upload files icon).

B 3: Compile

- Solidity Compiler icon. Choose compiler 0.8.19 or newer. Click **Compile StudentManager.sol**. A green tick means success and there should be no warnings.

B 4: Connect MetaMask

- Deploy and Run Transactions icon. Environment: **Injected Provider - MetaMask**. Approve the connection popup.
- Confirm Remix shows Sepolia (11155111) and your account with balance.

B 5: Deploy and capture the first gas data

- Contract: **StudentManager**. Value 0. Click **Deploy**.
- In the MetaMask popup note the **estimated network fee** and screenshot it. Click Confirm.
- Wait for success in the Terminal. Expand the creation transaction line and note gas, transaction cost and execution cost.
- Copy the contract address from Deployed Contracts.

Phase C: Test structures and arrays

In Remix, a function with several inputs has one text box. Type the values separated by commas, text in double quotes.

#	Action	Expected result
1	addStudent: `"Vedant", 21, "Computer Engineering"`. Confirm.	Success. Log StudentAdded with id 1.
2	addStudent: `"Aman", 20, "IT"`. Confirm.	Success, id 2.
3	addStudent: `"Riya", 22, "Mechanical"`. Confirm.	Success, id 3.
4	Click blue buttons studentCount and getStudentsLength.	Both return 3.
5	getStudent with input 1.	Tuple: 1, Vedant, 21, Computer Engineering.
6	getAllStudents.	An array with the 3 students.
7	updateStudent: `1, "Vedant S", 22, "Computer Engg"`. Confirm. Then getStudent 1.	Shows the new values.
8	removeStudent with input 1. Confirm. Then getAllStudents.	Array now holds id 3 Riya first and id 2 Aman second (swap and pop). studentCount is still 3.
9	getStudent with input 1.	Reverts: Student not found.

Phase D: Test the fallback and receive functions

Scroll to the bottom of the deployed contract in Remix to the section **Low level interactions**. It has a **CALLDATA** box and a **Transact** button.

#	Action	Expected result
1	Value = 0.001 Ether. Click the red **deposit** button. Confirm. Set Value to 0.	depositsCount = 1, lastSender = your address, lastDepositAmount = 1000000000000000. Log Received.
2	Value = 0.001 Ether. **CALLDATA left empty**. Click **Transact**. Confirm. Then read depositsCount.	**receive()** ran. Log Received. depositsCount = 2.
3	Value = 0. **CALLDATA = 0x1234**. Click **Transact**. Confirm.	**fallback()** ran because 0x1234 matches no function. Log **FallbackCalled** with data 0x1234. depositsCount = 3.
4	Read lastSender, lastDepositAmount, depositsCount.	Values match the last call.

TIP: Where to see the event: in the Remix Terminal expand the transaction and look at **logs**, or open the transaction on Etherscan and click the **Logs** tab. Received means receive() or deposit() ran. FallbackCalled means fallback() ran. This is your proof of the fallback.

WARNING: CALLDATA must be hexadecimal starting with 0x. Reset Value to 0 after each payable test.

Phase E: Observe the transaction fee and gas

Collect these numbers for the deployment and for each action. Three places show them:

Where	What you see
MetaMask confirm popup	Estimated network fee before you confirm (an estimate, the final fee is lower or equal).
Remix Terminal (expand the transaction)	Gas limit, transaction cost and execution cost in gas units.
Etherscan (sepolia.etherscan.io, open the transaction)	Transaction Fee (ETH), Gas Price (gwei), Gas Limit and Usage by Txn (gas used and percentage).

Transaction cost in Remix includes the 21,000 base gas, the data cost and the execution. **Execution cost** is only the EVM work. On Etherscan, gas used is the number to put in the table.

Your observation table (fill from your own transactions):

Operation	Gas used	Gas price (gwei)	Fee (ETH)	Tx hash (first 10 chars)
Deploy StudentManager				
addStudent (first)				
addStudent (second)				
updateStudent				
removeStudent				
deposit (first)				
receive (plain ETH)				
fallback (0x1234)				

3.4 Reference gas figures for checking your results

Measured by running this exact compiled contract on a local Ethereum test simulator, in the same order as the tests above. Sepolia values will be close, not identical. The fee column uses an **assumed** price of 2 gwei for illustration.

Operation	Gas used	Fee at 2 gwei (ETH)	Why
Deploy StudentManager	1,520,848	0.003041696	Stores about 6.8 KB of contract code at 200 gas per byte, plus 53,000 base and data costs.
addStudent (first)	182,665	0.00036533	Writes about seven empty slots, including the counter and array length.
addStudent (second)	148,225	0.00029645	Counter and array length are already non-zero, so about 34,000 less.
addStudent (third)	148,321	0.000296642	Same as the second.
updateStudent	45,589	0.000091178	Overwrites existing slots only.
removeStudent	64,079	0.000128158	Moves one element, clears the last slot (refund).
deposit (first call)	89,429	0.000178858	Three empty slots filled for the first time.
receive (after deposit)	32,314	0.000064628	Slots are already non-zero. If receive is your first call it costs close to the first deposit.
fallback (0x1234, 0 ETH)	31,423	0.000062846	Same reason.
View calls	0, free	0	Read-only, nothing is written.

EXAM: What to conclude in your journal: deployment costs the most because the contract code is stored on the chain. Functions that write new data cost more than functions that update existing data. Read-only functions are free. Fee = gas used x gas price, so the same transaction costs more when the network is busy. Plain ETH transfer is the baseline at 21,000 gas.

3.5 Troubleshooting

Problem	Cause	Fix
Compile error	Partial paste or old compiler	Paste the whole file, compiler 0.8.19 or newer.
Input rejected for addStudent	Text not in double quotes, or age above 255	Use `"Vedant", 21, "IT"` format. Age must be 0 to 255.
Remix says the transaction will fail	Calling a function with a wrong id, or Value above 0 on a non-payable function	Check the id exists. Set Value to 0.
Low level Transact fails with empty CALLDATA	Value is 0 and nothing happens, or Value unit wrong	Set Value above 0 for the receive test.
CALLDATA error	Missing 0x or odd number of hex digits	Use 0x1234.
Fallback not triggered	CALLDATA matches a real function selector	Use short arbitrary data such as 0x1234.
Gas used differs from this guide	Network rules or compiler settings differ, or first-write effect	Normal. Compare relative sizes, not exact digits.

3.6 Viva questions

Question	Answer
What is a struct?	A user-defined type that groups fields, like a Java class with only data.
Array versus mapping?	An array is ordered and has a length, a mapping is a hash lookup with no length. This contract uses both: the array for the list, the mapping for fast lookup.
Why does idx store position + 1?	A mapping returns 0 for missing keys. Storing +1 lets 0 mean 'not found'.
How is removal done?	Swap and pop: move the last element into the hole, update its index, pop the last slot. Constant gas, order not kept.

Question	Answer
storage vs memory vs calldata?	storage is permanent and costly, memory is temporary during a call, calldata is read-only call input and the cheapest.
What is a fallback function?	A function that runs when no function matches the call data. With payable it can also accept ETH.
receive vs fallback?	receive handles plain ETH with empty data. fallback handles unmatched data (and ETH with empty data if there is no receive).
How did you prove fallback ran?	I sent CALLDATA 0x1234 and saw the FallbackCalled event with that data, and depositsCount increased.
What is msg.data?	The raw call data bytes of the current call.
What is gas and who pays?	The unit of computation cost. The sender of the transaction pays gas used x gas price in ETH.
Why is deployment costly?	The contract code is stored on-chain at 200 gas per byte and the data is paid per byte.
Why does the first addStudent cost more?	It writes storage slots that were empty (about 22,100 gas each). Later calls overwrite non-zero slots (about 5,000).
Why are view functions free?	They only read data on a node and send no transaction.
What is indexed in an event?	The parameter becomes a topic, so it can be searched and filtered.
What does the constructor do?	Runs once at deployment. Here it sets studentCount to 0.
What is uint8?	An 8-bit unsigned integer, 0 to 255.

Appendix: Quick Reference

A. Exam-day checklist

- Charged laptop or a lab PC with internet and a supported browser.
- 12-word recovery phrase on paper, only if you plan to restore an existing practice wallet.
- Test ETH: at least 0.05 SepoliaETH in the wallet, claimed the day before. Faucets can be slow or limited.
- The .sol files ready to paste: 3_Bank_account.sol and 4_Student.sol.
- Notebook or doc for screenshots: MetaMask, compile success, deployment popup, each function result, Etherscan pages.

B. Unit converter

Ether	Wei
1	1000000000000000000
0.1	100000000000000000
0.01	10000000000000000
0.005	5000000000000000
0.001	1000000000000000

C. Remix button colours

Colour	Function type	Costs gas?
Blue	view / pure (read only)	No when called from outside
Orange	State-changing, not payable	Yes, needs MetaMask confirmation
Red	payable (can receive ETH)	Yes, set the Value field first

D. Common revert messages in these contracts

Message	Contract	Meaning
Account exists	BankAccount	createAccount called twice by the same address
No account	BankAccount	Caller never opened an account
Amount > 0	BankAccount	Deposit of zero
Low balance	BankAccount	Withdraw more than the balance
Student not found	StudentManager	Id does not exist or was removed
Send ETH	StudentManager	deposit() with Value 0

E. Useful links (type them yourself, do not follow ads)

- remix.ethereum.org (Remix IDE)
- metamask.io (MetaMask)
- sepolia.etherscan.io (Sepolia block explorer)
- alchemy.com/faucets/ethereum-sepolia (one Sepolia faucet; others in Part 1)